



NATIONAL UNIVERSITY OF MODERN LANGUAGES

Artificial Intelligence Final Project

NAME: M Abdul Rafay (FL23791)

M Usman Tanveer (FL23848)

Elisha Abbasi (SP23606)

Zulkifal (SP23660)

PROGRAM: BSSE (6th Semester)

COURSE: Artificial Intelligence

SUBMITTED TO: Mam Amna Sajjad

DATE: 5/15/2026

DAY: Friday

Table Of Contents

1. Introduction.....	1
2. Dataset Overview.....	1
3. Task 1: Dataset Loading and Exploration	2
3.1 First 20 Rows	2
3.2 Shape, Column Names, and Data Types	3
3.3 Missing Values	4
3.4 Summary Statistics.....	4
3.5 What This Dataset Is About	5
4. Task 2: Problem Type.....	5
5. Task 3: Data Preprocessing	6
5.1 Handling Missing Values	6
5.2 Encoding Categorical Columns	6
5.3 Feature and Target Separation.....	6
5.4 Train-Test Split.....	7
5.5 Feature Scaling.....	7
6. Task 4: Data Visualization	7
6.1 Correlation Heatmap.....	7
6.2 Boxplot.....	8
6.3 Histogram.....	9
6.4 Scatter Plot.....	9
6.5 Bar Chart.....	10
6.6 Pie Chart.....	11
7. Task 5: Machine Learning Algorithms.....	11
7.1 Logistic Regression.....	11
7.2 Random Forest Classifier.....	11
7.3 K-Nearest Neighbors ($k = 7$).....	12
8. Task 6: Model Evaluation	12
8.1 Logistic Regression Results	12
8.2 Random Forest Results	13
8.3 K-Nearest Neighbors Results.....	14
9. Comparison of Models.....	15
10. Result Interpretation.....	15
10.1 Best Model	15
10.2 Observations from the Data	15
10.3 Three Insights from the Visualizations.....	16
10.4 Limitations and Possible Improvements	16

11. Conclusion	16
References.....	17
Appendix: Python Code.....	18

List of Figures

Figure 1: First 20 rows of the dataset — <code>df.head(20)</code>	3
Figure 2: <code>df.dtypes</code> output	3
Figure 3: <code>df.info()</code> output showing non-null counts for all columns	4
Figure 4: <code>df.isnull().sum()</code> — zero missing values in all columns	4
Figure 5: <code>df.describe()</code> — summary statistics for all numerical columns	5
Figure 6: Preprocessing confirmation — encoding output, split sizes, and scaling	7
Figure 7: Correlation Heatmap of all numerical features	7
Figure 9: Boxplot of Stress Level grouped by Depression Label	8
Figure 8: Histogram of Daily Social Media Hours	9
Figure 10: Scatter Plot of Sleep Hours vs Academic Performance	9
Figure 11: Bar Chart of Platform Usage	10
Figure 12: Pie Chart of Depression Label Distribution	11
Figure 13: Logistic Regression — Accuracy, F1 Score, and Classification Report	12
Figure 14: Confusion Matrix Heatmap — Logistic Regression	12
Figure 15: Full notebook running in Google Colab. all 16 cells executed	26

List of Tables

Table 1: Dataset Columns, Data Types, and Descriptions	1
Table 2: Comparison of Problem Types	6
Table 3: Model Comparison Summary	15

1. Introduction

This lab project applies machine learning techniques on the Teen Mental Health Dataset to predict whether a teenager is at risk of depression. The dataset contains 1,200 records and 13 features related to social media usage, sleep, stress, anxiety, academic performance, and physical activity.

The task is a binary classification problem because the target variable, `depression_label`, contains two classes: 0 (not at risk) and 1 (at risk). The project was implemented in Python using Google Colab. The workflow included dataset exploration, preprocessing, visualization, model training, and evaluation.

2. Dataset Overview

The dataset contains 1,200 rows and 13 columns with no missing values. Features include age, gender, daily social media usage, sleep hours, stress level, anxiety level, addiction level, and academic performance.

The target variable `depression_label` is highly imbalanced:

- Class 0 (Not Depressed): 1,169 samples (97.42%)
- Class 1 (Depressed): 31 samples (2.58%)

This imbalance affects model performance and evaluation.

Table 1: Dataset Columns, Data Types, and Descriptions

Column Name	Type	What it stores
age	int64	Age of the student (13 to 19)
gender	object	Gender of the student — male or female
daily_social_media_hours	float64	Hours per day spent on social media (1.0 to 8.0)
platform_usage	object	Which platform the student uses: Instagram, TikTok, or Both

Column Name	Type	What it stores
sleep_hours	float64	Average number of hours of sleep per night
screen_time_before_sleep	float64	Hours of screen time immediately before sleeping
academic_performance	float64	Academic score on a GPA-style scale from 2.0 to 4.0
physical_activity	float64	Hours of physical activity per day (0.0 to 2.0)
social_interaction_level	object	How socially active the student is: low, medium, or high
stress_level	int64	Self-reported stress score on a scale of 1 to 10
anxiety_level	int64	Self-reported anxiety score on a scale of 1 to 10
addiction_level	int64	Social media addiction score on a scale of 1 to 10
depression_label	int64	Target variable — 0: not at risk, 1: at risk of depression

3. Task 1: Dataset Loading and Exploration

3.1 First 20 Rows

The dataset was loaded using `pd.read_csv()`. We then printed the first 20 rows with `df.head(20)` to check that the data loaded correctly and to get a general sense of what the values look like.

```

Shape: (1200, 13)
   age  gender  daily_social_media_hours  platform_usage  sleep_hours  screen_time_before_sleep  academic_performance  physical_activity
0    14   male         7.9             Instagram         7.4             2.9             3.01             1.5
1    19  female         1.9             TikTok          8.0             2.9             3.22             0.8
2    17  female         1.3             Instagram         7.6             0.5             3.92             0.0
3    15   male         7.4             TikTok          6.9             1.6             3.48             0.8
4    15  female         4.7             Both          4.9             3.0             2.37             1.4
5    19  female         7.4             Both          4.4             2.4             2.63             0.6
6    18  female         2.5             Instagram         6.4             2.4             2.63             0.7
7    16   male         4.0             Both          4.2             0.5             2.40             1.3
8    19  female         3.3             TikTok          5.0             2.1             2.04             0.9
9    15   male         1.9             TikTok          4.9             1.5             3.77             1.1
10   16   male         7.2             Both          5.8             2.9             3.16             1.2
11   13   male         4.9             Instagram         4.7             2.5             3.92             1.2
12   16   male         6.2             Both          7.2             2.6             2.02             1.7
13   17  female         6.8             TikTok          5.7             1.1             3.29             1.3
14   15   male         5.7             TikTok          5.9             0.8             3.50             0.7
15   14  female         3.6             Both          5.3             0.6             2.99             0.3
16   13   male         1.5             Both          4.2             1.4             2.59             0.5
17   16   male         3.1             Both          6.1             0.8             2.11             1.9
18   16  female         6.7             Both          6.8             1.9             3.08             1.4
19   14  female         1.2             Instagram         7.6             1.0             3.85             0.8

```

```

social_interaction_level  stress_level  anxiety_level  addiction_level  depression_label
low                      2                2                1                0
high                    8                1               10                0
high                    2                4                2                0
medium                  1                7                9                0
medium                  3                5                2                0
high                    3                5                7                0
low                     2                2                5                0
low                     6               10                5                0
high                    1               10                9                0
high                    1                1                4                0
medium                  5                8                9                0
low                     1               10                8                0
medium                  5                7                6                0
medium                  1               10               10                0
low                     1                9                1                0
high                    4               10                2                0
high                    4                5                8                0
high                   10                7               10                0
high                   10                9                1                0
high                    8                4                6                0

```

Figure 1: First 20 rows of the dataset — df.head(20)

3.2 Shape, Column Names, and Data Types

Running df.shape confirmed the dataset has 1,200 rows and 13 columns. We used df.dtypes to see the data type of each column and df.info() to check that all columns are fully populated with no null entries.

```

age                int64
gender             object
daily_social_media_hours  float64
platform_usage      object
sleep_hours         float64
screen_time_before_sleep  float64
academic_performance  float64
physical_activity    float64
social_interaction_level  object
stress_level        int64
anxiety_level       int64
addiction_level     int64
depression_label    int64
dtype: object

```

Figure 2: df.dtypes output

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 13 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   age                                    1200 non-null   int64
1   gender                                1200 non-null   object
2   daily_social_media_hours              1200 non-null   float64
3   platform_usage                        1200 non-null   object
4   sleep_hours                           1200 non-null   float64
5   screen_time_before_sleep              1200 non-null   float64
6   academic_performance                  1200 non-null   float64
7   physical_activity                     1200 non-null   float64
8   social_interaction_level              1200 non-null   object
9   stress_level                          1200 non-null   int64
10  anxiety_level                         1200 non-null   int64
11  addiction_level                       1200 non-null   int64
12  depression_label                      1200 non-null   int64
dtypes: float64(5), int64(5), object(3)
memory usage: 122.0+ KB
None

```

Figure 3: df.info() output showing non-null counts for all columns

3.3 Missing Values

We ran `df.isnull().sum()` and the result was 0 for every single column. The dataset is complete and no filling or removal of rows was needed.

```

age                                0
gender                             0
daily_social_media_hours           0
platform_usage                     0
sleep_hours                        0
screen_time_before_sleep           0
academic_performance               0
physical_activity                   0
social_interaction_level            0
stress_level                       0
anxiety_level                      0
addiction_level                    0
depression_label                   0
dtype: int64

```

Figure 4: `df.isnull().sum()` — zero missing values in all columns

3.4 Summary Statistics

`df.describe()` showed:

- Average age: 15.93 years
- Average social media usage: 4.54 hours/day
- Average sleep: 6.45 hours

- Average stress level: 5.45
- Average anxiety level: 5.64

	age	daily_social_media_hours	sleep_hours	screen_time_before_sleep	academic_performance
count	1200.000000	1200.000000	1200.000000	1200.000000	1200.000000
mean	15.928333	4.536667	6.449417	1.740333	2.990383
std	2.021947	2.029599	1.442677	0.716660	0.576758
min	13.000000	1.000000	4.000000	0.500000	2.000000
25%	14.000000	2.800000	5.200000	1.100000	2.500000
50%	16.000000	4.500000	6.500000	1.800000	2.990000
75%	18.000000	6.300000	7.600000	2.400000	3.480000
max	19.000000	8.000000	9.000000	3.000000	4.000000


```

Class distribution:
depression_label
0    1169
1      31
Name: count, dtype: int64

Class distribution (%):
depression_label
0    97.416667
1     2.583333
Name: proportion, dtype: float64

```


physical_activity	stress_level	anxiety_level	addiction_level	depression_label
1200.000000	1200.000000	1200.000000	1200.000000	1200.000000
1.014500	5.445833	5.636667	5.565000	0.025833
0.582185	2.903290	2.859453	2.830627	0.158704
0.000000	1.000000	1.000000	1.000000	0.000000
0.500000	3.000000	3.000000	3.000000	0.000000
1.000000	5.000000	6.000000	6.000000	0.000000
1.500000	8.000000	8.000000	8.000000	0.000000
2.000000	10.000000	10.000000	10.000000	1.000000

Figure 5: df.describe() — summary statistics for all numerical columns

3.5 What This Dataset Is About

This dataset was created to examine the relationship between teenage social media behaviour and mental health outcomes. Each row represents one student. Most of the scores such as stress, anxiety, and addiction are self-reported, meaning the values depend on how the student rated themselves. The target column `depression_label` is binary. Because 97.42% of the rows belong to class 0, a model that simply always predicts 0 would still achieve over 97% accuracy. This is why we also look at the weighted F1-score rather than relying on accuracy alone.

4. Task 2: Problem Type

Decision: Binary Classification

The column we are predicting, `depression_label`, has exactly two possible values: 0 and 1. This makes it a classification problem, not regression and not clustering.

We would choose regression if the target were a continuous number, such as predicting an actual anxiety score or a GPA value. We would choose clustering if there were no label column and we had to group the students ourselves based on their similarities. Neither applies here. The label already exists and it is categorical with two classes. Table 2 summarises this comparison.

Table 2: Comparison of Problem Types

Type	When to use	Applies here?
Classification	Target is a category or label	Yes depression_label is 0 or 1
Regression	Target is a continuous numerical value	No we are not predicting a number
Clustering	No label exists; data is grouped freely	No the label column already exists

5. Task 3: Data Preprocessing

5.1 Handling Missing Values

As confirmed in the exploration step, the dataset has no missing values anywhere. No imputation or row removal was necessary.

5.2 Encoding Categorical Columns

Categorical columns were converted into numeric form:

- Gender: female = 0, male = 1
- Social interaction: low = 0, medium = 1, high = 2
- Platform usage: Instagram = 0, TikTok = 1, Both = 2

5.3 Feature and Target Separation

After encoding, all 12 input columns were stored as the feature matrix X. The depression_label column was stored separately as y, the target variable we want to predict.

5.4 Train-Test Split

The data was split 80/20 using `train_test_split()` with `random_state=42` and `stratify=y`. The `stratify` parameter makes sure the same class proportion (97.42% class 0 and 2.58% class 1) is maintained in both the training and test sets.

- Training set: 960 samples (25 depressed, 935 not depressed)
- Test set: 240 samples (6 depressed, 234 not depressed)

5.5 Feature Scaling

`StandardScaler` was applied for Logistic Regression and KNN because these algorithms depend on feature magnitude. Random Forest was trained on unscaled data.

Encoding done.						
	gender	gender_enc	social_interaction_level	social_enc	platform_usage	platform_enc
0	male	1	low	0	Instagram	0
1	female	0	high	2	TikTok	1
2	female	0	high	2	Instagram	0
3	male	1	medium	1	TikTok	1
4	female	0	medium	1	Both	2

Figure 6: Preprocessing confirmation — encoding output, split sizes, and scaling

6. Task 4: Data Visualization

6.1 Correlation Heatmap

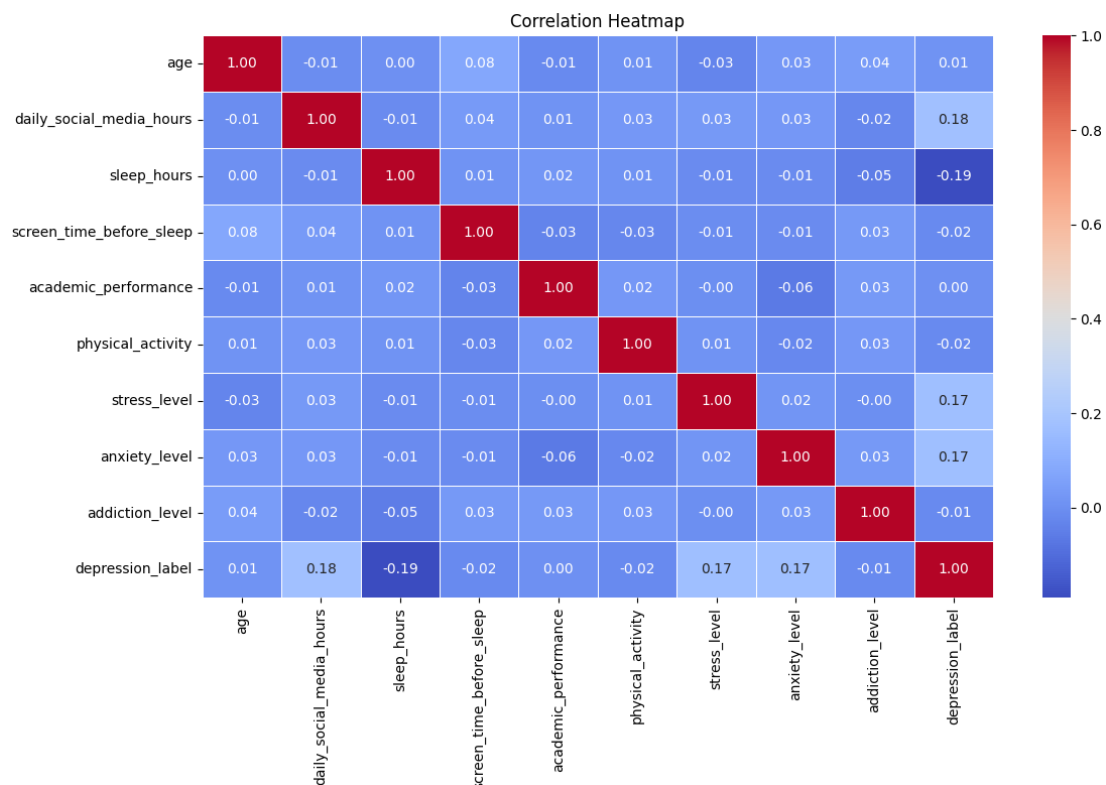


Figure 7: Correlation Heatmap of all numerical features

The heatmap shows the pairwise correlation between all numerical columns. The strongest positive correlation is between `stress_level` and `anxiety_level`, which makes sense as students who report high stress tend to also report high anxiety. Addiction level and daily social media hours are also positively linked. The correlation between individual features and the target column `depression_label` is generally low, meaning no single feature by itself is a strong predictor of depression in this dataset.

6.2 Boxplot

The boxplot compares the distribution of `stress_level` between the two groups. Students labelled as depressed (class 1) have a noticeably higher median stress level compared to students who are not depressed (class 0). The spread of values is also wider for the depressed group. Among all the features we visualized, stress level showed the clearest difference between the two classes.

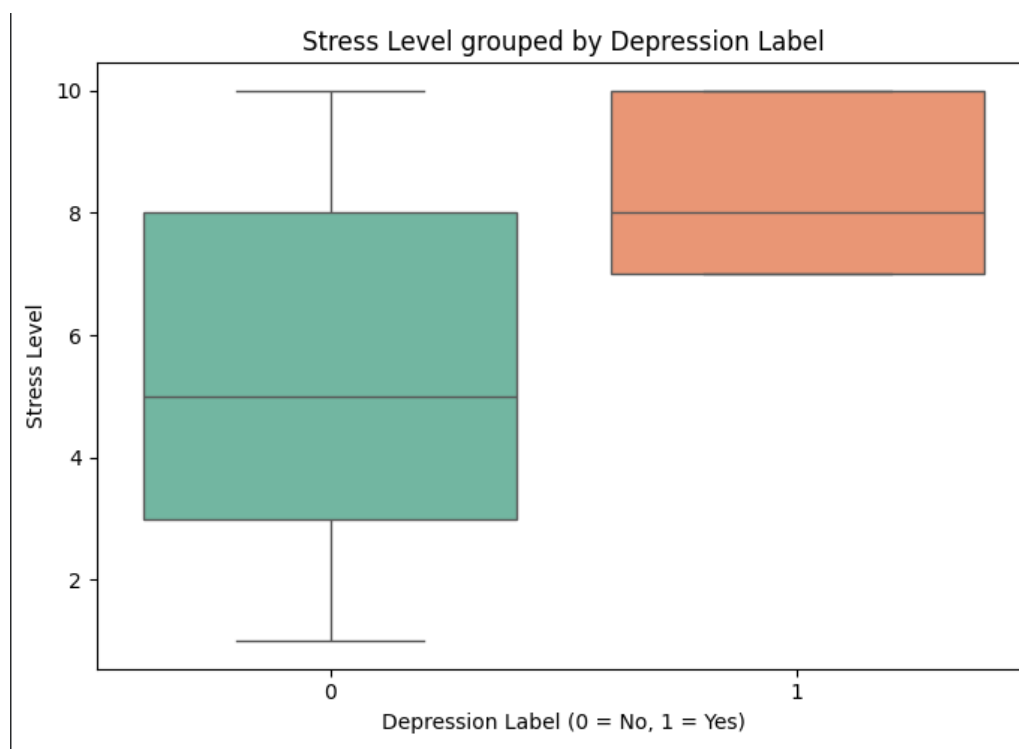


Figure 8: Boxplot of Stress Level grouped by Depression Label

6.3 Histogram

The distribution of daily social media hours is fairly even across the range of 1 to 8 hours. There is no strong peak at any one value. This tells us the dataset does not heavily favour light or heavy social media users and provides a good spread of usage patterns for the model to learn from.

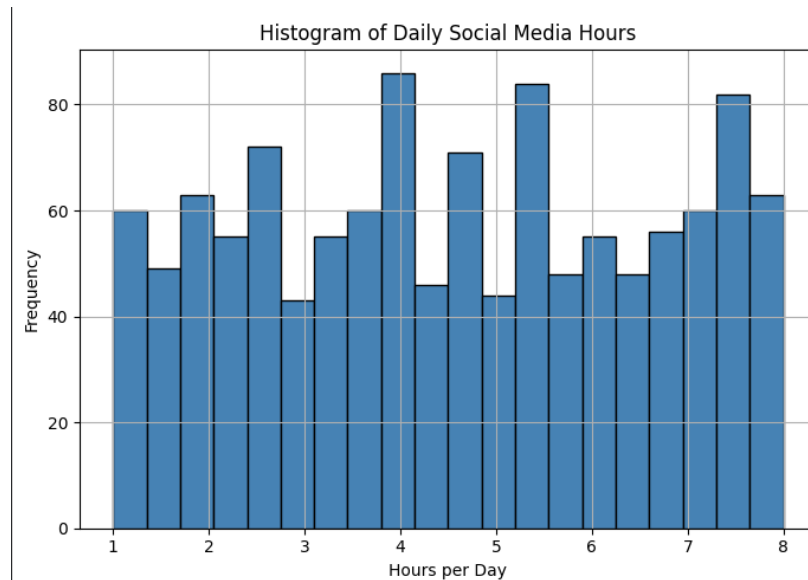


Figure 9: Histogram of Daily Social Media Hours

6.4 Scatter Plot

The scatter plot shows sleep hours on the x-axis and academic performance on the y-axis, with the two classes coloured differently. Students who sleep less than 5 hours appear slightly more toward the lower end of academic performance, but the trend is weak. More importantly, the

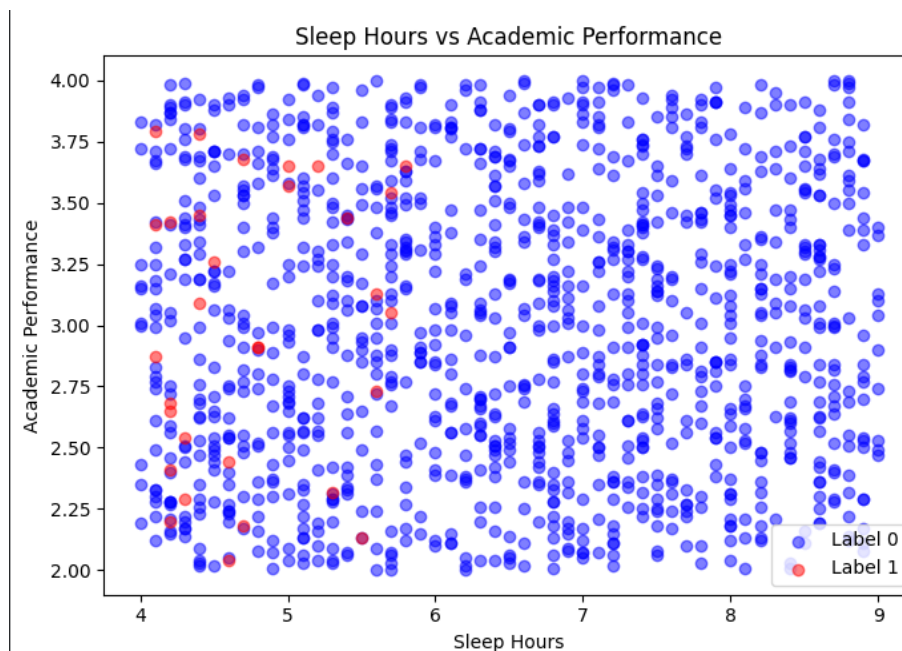


Figure 10: Scatter Plot of Sleep Hours vs Academic Performance

depressed students (class 1) do not form a separate cluster anywhere on the plot. They are scattered throughout the same regions as non-depressed students, which is one reason why classification is so difficult on this dataset.

6.5 Bar Chart

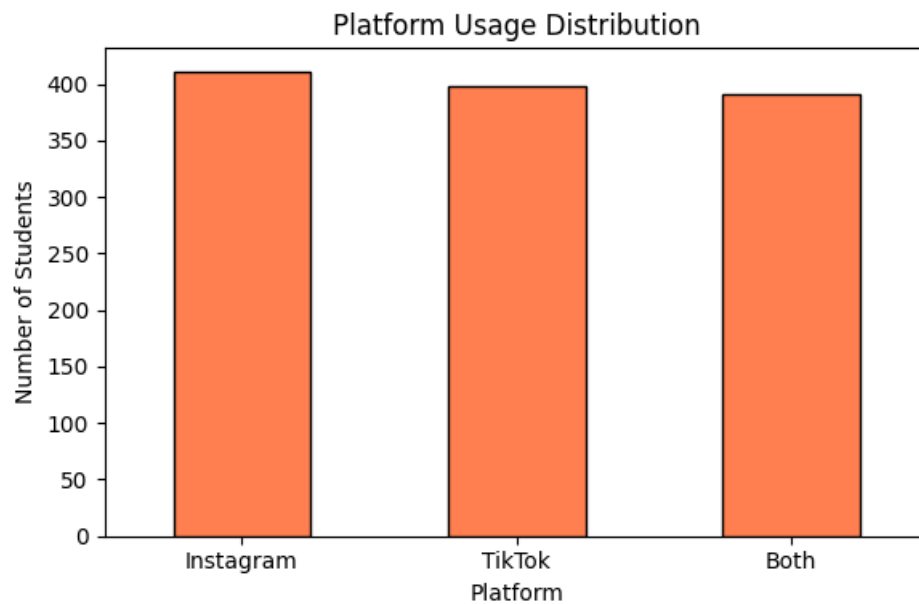


Figure 11: Bar Chart of Platform Usage

The bar chart shows that the three platform categories have nearly equal representation: Instagram has 411 students, TikTok has 398, and Both has 391. This balanced distribution means platform usage is unlikely to introduce any bias into the model.

6.6 Pie Chart

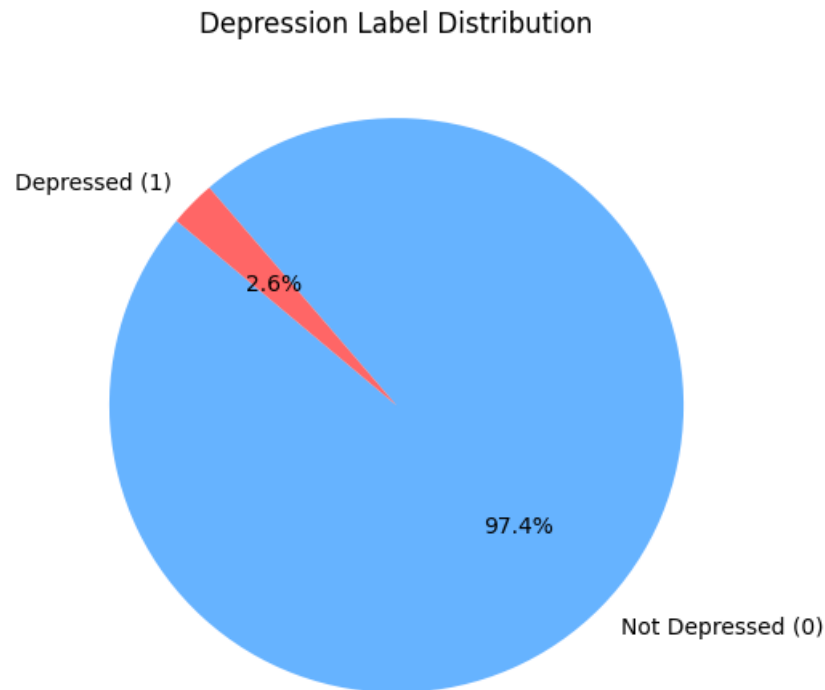


Figure 12: Pie Chart of Depression Label Distribution

The pie chart clearly shows a severe class imbalance, with 97.42% of students not depressed and only 2.58% depressed. Because of this imbalance, a model that always predicts class 0 can still achieve over 97% accuracy without identifying any depressed students. This makes accuracy an unreliable evaluation metric for this dataset.

7. Task 5: Machine Learning Algorithms

Three classification algorithms were applied to the dataset.

7.1 Logistic Regression

Logistic Regression is a linear classification model that estimates the probability of a data point belonging to each class and then assigns the class with the higher probability. It is a standard starting point for binary classification and the results are straightforward to interpret. We set `max_iter=1000` to ensure the solver converges properly given the number of features and the nature of the data. This model was trained on the scaled feature set.

7.2 Random Forest Classifier

Random Forest builds an ensemble of 100 decision trees, each trained on a random sample of the data, and combines their predictions by majority vote. It can capture non-linear

relationships between features and is generally more robust to irrelevant features than a single decision tree. Since tree-based models split on value thresholds rather than distances, scaling is not required. We used `n_estimators=100` and `random_state=42`, and trained the model on the original unscaled feature matrix.

7.3 K-Nearest Neighbors (k = 7)

KNN classifies a new data point by finding its k nearest neighbours in the training set and assigning whichever label the majority of those neighbours have. We chose $k = 7$ because using an odd number prevents tied votes and 7 is a reasonable default for a dataset of this size. KNN relies entirely on distance between data points, so it requires scaled features. This model was trained on the scaled training set.

8. Task 6: Model Evaluation

8.1 Logistic Regression Results

```

=== Logistic Regression Results ===
Accuracy : 0.9875
F1 Score : 0.9855
Confusion Matrix:
[[234  0]
 [ 3  3]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.99	1.00	0.99	234
1	1.00	0.50	0.67	6
accuracy			0.99	240
macro avg	0.99	0.75	0.83	240
weighted avg	0.99	0.99	0.99	240

Figure 13: Logistic Regression — Accuracy, F1 Score, and Classification Report



Figure 14: Confusion Matrix Heatmap — Logistic Regression

Logistic Regression achieved an accuracy of **98.75%** and a weighted F1-score of **0.9855**. The confusion matrix showed 234 true negatives, 0 false positives, 3 false negatives, and 3 true positives. This means it correctly identified 3 out of the 6 depressed students in the test set, giving a recall of 50% for class 1. It was the only model that detected any depressed students at all.

8.2 Random Forest Results

```

=== Random Forest Results ===
Accuracy : 0.9750
F1 Score : 0.9627
Confusion Matrix:
[[234  0]
 [ 6  0]]
Classification Report:

```

		precision	recall	f1-score	support
	0	0.97	1.00	0.99	234
	1	0.00	0.00	0.00	6
accuracy				0.97	240
macro avg		0.49	0.50	0.49	240
weighted avg		0.95	0.97	0.96	240

Figure 14: Random Forest — Accuracy, F1 Score, and Classification Report

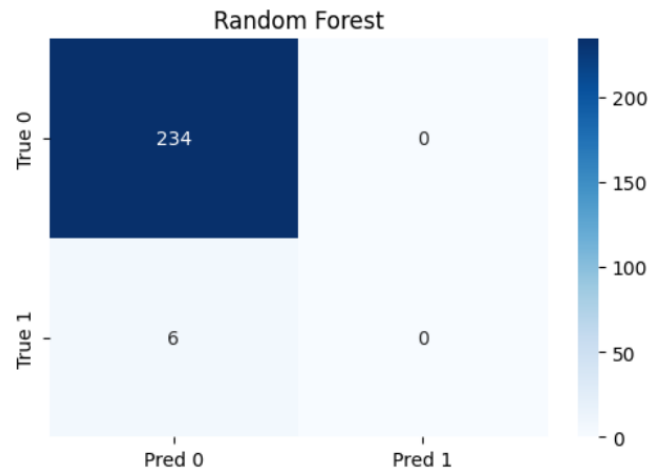


Figure 15: Confusion Matrix Heatmap — Random Forest

Random Forest achieved an accuracy of **97.50%** and a weighted F1-score of **0.9627**. The confusion matrix showed 234 true negatives, 0 false positives, 6 false negatives, and 0 true positives. It predicted class 0 for every single test sample and did not detect any depressed student. The high accuracy comes purely from the class imbalance, not from the model learning anything useful about the minority class.

8.3 K-Nearest Neighbors Results

```
=== KNN Results (k=7) ===  
Accuracy : 0.9750  
F1 Score : 0.9627  
Confusion Matrix:  
[[234  0]  
 [ 6  0]]  
Classification Report:  
              precision    recall  f1-score   support  
  
      0       0.97       1.00       0.99       234  
      1       0.00       0.00       0.00        6  
  
   accuracy       0.97       0.97       0.97       240  
  macro avg       0.49       0.50       0.49       240  
weighted avg       0.95       0.97       0.96       240
```

Figure 16: K-Nearest Neighbors. Accuracy, F1 Score, and Classification Report

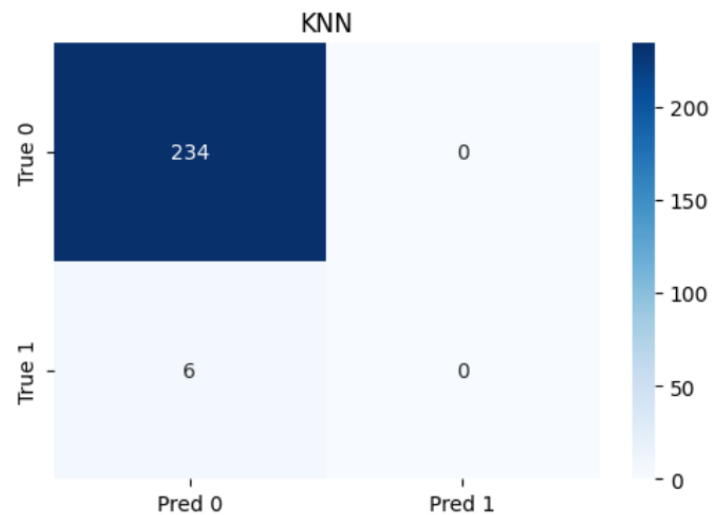


Figure 17: Confusion Matrix Heatmap. KNN

KNN also achieved **97.50%** accuracy and a weighted F1-score of **0.9627**. Its confusion matrix was identical to Random Forest: 234 true negatives, 0 false positives, 6 false negatives, 0 true positives. When the training data has very few minority-class examples, the 7 nearest neighbours of any test point are almost always class 0, so the model ends up always predicting 0.

9. Comparison of Models

Table 3: Model Comparison Summary

Model	Accuracy	F1 (Weighted)	Comment
Logistic Regression	98.75%	0.9855	Best overall correctly identified 3 out of 6 depressed students (recall 50%)
Random Forest	97.50%	0.9627	Predicted class 0 for every test sample — zero depressed students detected
K-Nearest Neighbors	97.50%	0.9627	Same outcome as Random Forest — no minority class detected

10. Result Interpretation

10.1 Best Model

Logistic Regression performed best in this experiment, correctly identifying 3 out of 6 depressed students. Unlike Random Forest and KNN, it was able to capture some minority-class patterns. This is because Logistic Regression learns a linear decision boundary in a scaled feature space, which can slightly favor minority class detection. In contrast, tree-based and distance-based models tend to bias toward the majority class (97.42%), especially under severe class imbalance.

10.2 Observations from the Data

The dataset was clean and required minimal preprocessing. The main challenge was extreme class imbalance, with only 31 depressed samples out of 1,200. As a result, Random Forest and KNN failed to detect any depressed cases despite achieving 97.50% accuracy, showing that accuracy is misleading in this context. A weighted F1-score provides a more realistic evaluation of model performance on imbalanced data.

10.3 Three Insights from the Visualizations

1. The correlation heatmap shows that *stress level* and *anxiety level* are the most strongly related features. Both also have a weak positive relationship with the depression label, but overall correlations are low.
2. The boxplot of stress level shows a clear difference between classes. Depressed students have higher median stress and greater variability, making stress the most distinguishing feature.
3. The scatter plot of sleep hours vs academic performance shows heavy overlap between both classes. There is no clear separation, which explains why even Logistic Regression only detected half of the depressed cases.

10.4 Limitations and Possible Improvements

The main limitation is severe class imbalance in the dataset. To improve performance, techniques such as class weighting in Logistic Regression, SMOTE oversampling, or collecting more minority-class data could be used. Advanced models like XGBoost with `scale_pos_weight` could also help handle imbalance more effectively.

11. Conclusion

This project covered a full machine learning workflow using the Teen Mental Health Dataset, including data exploration, visualization, preprocessing, model training, and evaluation.

The key takeaway is that accuracy alone is misleading in imbalanced datasets. Although Random Forest and KNN achieved 97.50% accuracy, they failed to identify any depressed students. Logistic Regression performed better by detecting 3 out of 6 cases, showing that evaluation metrics like F1-score and recall are more meaningful for this type of problem.

Project Source Code (GitHub Repository):

GitHub:<https://github.com/git-rafaykhan/university-resources/tree/17a5316ce1ff6fb49d201b27b37a511cd6604625/sem6/ai/project>

References

1. [Teen Mental Health Dataset](#)
2. [Scikit-learn: Machine Learning in Python](#)
3. [Data Structures for Statistical Computing in Python](#)
4. [Seaborn: Statistical Data Visualization](#)
5. [Matplotlib: A 2D Graphics Environment](#)

Appendix: Python Code

The complete code was written across 16 cells in Google Colab. Each cell is shown below in the order it was run.

Notebook Link:

<https://colab.research.google.com/drive/1O-A77ASsOVUkPbJWKKZn5oYEjGny1A3D?usp=sharing>

Block 1 — Import Libraries

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import RandomForestClassifier

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (accuracy_score, f1_score,
                             confusion_matrix, classification_report)

import warnings

warnings.filterwarnings('ignore')
```

Block 2 — Load Dataset

```
df = pd.read_csv('Teen_Mental_Health_Dataset.csv')

print('Shape:', df.shape)

print(df.head(20))
```

Block 3 — Column Data Types

```
print(df.dtypes)
```

Block 4 — Dataset Info

```
print(df.info())
```

Block 5 — Missing Values Check

```
print(df.isnull().sum())
```

Block 6 — Summary Statistics

```
print(df.describe())
```

Block 7 — All Six Visualizations

```
# --- Figure 1: Correlation Heatmap ---

plt.figure(figsize=(12, 8))

sns.heatmap(df.corr(numeric_only=True), annot=True, fmt='.2f',
            cmap='coolwarm', linewidths=0.5)

plt.title('Correlation Heatmap')

plt.tight_layout()

plt.show()

# --- Figure 2: Boxplot - Stress Level by Depression Label ---

plt.figure(figsize=(7, 5))

sns.boxplot(x='depression_label', y='stress_level', data=df, palette='Set2')

plt.title('Stress Level by Depression Label')

plt.xlabel('Depression Label (0 = Not at Risk, 1 = At Risk)')

plt.ylabel('Stress Level')

plt.tight_layout()

plt.show()
```

```

# --- Figure 3: Histogram - Daily Social Media Hours ---

plt.figure(figsize=(7, 5))

df['daily_social_media_hours'].hist(bins=20, color='steelblue', edgecolor='black')

plt.title('Distribution of Daily Social Media Hours')

plt.xlabel('Hours per Day')

plt.ylabel('Number of Students')

plt.tight_layout()

plt.show()


# --- Figure 4: Scatter Plot - Sleep Hours vs Academic Performance ---

plt.figure(figsize=(7, 5))

for label, colour in zip([0, 1], ['steelblue', 'red']):

    subset = df[df['depression_label'] == label]

    plt.scatter(subset['sleep_hours'], subset['academic_performance'],

                label=f'Label {label}', alpha=0.5, color=colour)

plt.title('Sleep Hours vs Academic Performance')

plt.xlabel('Sleep Hours')

plt.ylabel('Academic Performance')

plt.legend()

plt.tight_layout()

plt.show()


# --- Figure 5: Bar Chart - Platform Usage ---

plt.figure(figsize=(6, 4))

df['platform_usage'].value_counts().plot(kind='bar',

    color='coral', edgecolor='black')

plt.title('Platform Usage Distribution')

plt.xlabel('Platform')

plt.ylabel('Number of Students')

```



```

plt.xticks(rotation=0)

plt.tight_layout()

plt.show()

# --- Figure 6: Pie Chart - Depression Label Distribution ---

plt.figure(figsize=(6, 5))

counts = df['depression_label'].value_counts()

plt.pie(counts, labels=['Not at Risk (0)', 'At Risk (1)'],

        autopct='%1.1f%%', startangle=140,

        colors=['#66b3ff', '#ff6666'])

plt.title('Depression Label Distribution')

plt.tight_layout()

plt.show()

```

Block 8 — Encoding Categorical Columns

```

le = LabelEncoder()

df['gender_enc'] = le.fit_transform(df['gender'])

df['social_enc'] = df['social_interaction_level'].map(

    {'low': 0, 'medium': 1, 'high': 2})

df['platform_enc'] = df['platform_usage'].map(

    {'Instagram': 0, 'TikTok': 1, 'Both': 2})

print('Encoding complete.')

print(df[['gender', 'gender_enc', 'social_interaction_level',

    'social_enc', 'platform_usage', 'platform_enc']].head(5))

```

Block 9 — Feature and Target Separation

```

feature_cols = ['age', 'gender_enc', 'daily_social_media_hours',

    'platform_enc', 'sleep_hours', 'screen_time_before_sleep',

    'academic_performance', 'physical_activity', 'social_enc',

```

```

        'stress_level', 'anxiety_level', 'addiction_level']

X = df[feature_cols]

y = df['depression_label']


print('Feature matrix shape:', X.shape)

print('Target shape:', y.shape)

print('Class distribution in y:')

print(y.value_counts())

```

Block 10 — Train-Test Split (80/20, stratified)

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)


print('Training set size:', X_train.shape)

print('Test set size: ', X_test.shape)

print('Class distribution in training set:')

print(y_train.value_counts())

print('Class distribution in test set:')

print(y_test.value_counts())

```

Block 11 — Feature Scaling with StandardScaler

```

scaler = StandardScaler()

X_train_sc = scaler.fit_transform(X_train)

X_test_sc = scaler.transform(X_test)


print('Scaling complete.')

print('Mean of scaled training features (should be ~0):')

print(X_train_sc.mean(axis=0).round(4))

```

Block 12 — Logistic Regression Training and Evaluation

```
lr_model = LogisticRegression(max_iter=1000, random_state=42)

lr_model.fit(X_train_sc, y_train)

y_pred_lr = lr_model.predict(X_test_sc)

print('=== Logistic Regression Results ===')

print(f'Accuracy : {accuracy_score(y_test, y_pred_lr):.4f}')

print(f'F1 Score : {f1_score(y_test, y_pred_lr,

                             average="weighted", zero_division=0):.4f}')

print()

print('Confusion Matrix:')

print(confusion_matrix(y_test, y_pred_lr))

print()

print('Classification Report:')

print(classification_report(y_test, y_pred_lr, zero_division=0))
```

Block 13 — Random Forest Training and Evaluation

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)

rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

print('=== Random Forest Results ===')

print(f'Accuracy : {accuracy_score(y_test, y_pred_rf):.4f}')

print(f'F1 Score : {f1_score(y_test, y_pred_rf,

                             average="weighted", zero_division=0):.4f}')

print()

print('Confusion Matrix:')

print(confusion_matrix(y_test, y_pred_rf))

print()
```

```
print('Classification Report:')

print(classification_report(y_test, y_pred_rf, zero_division=0))
```

Block 14 — K-Nearest Neighbors Training and Evaluation

```
knn_model = KNeighborsClassifier(n_neighbors=7)

knn_model.fit(X_train_sc, y_train)

y_pred_knn = knn_model.predict(X_test_sc)

print('=== K-Nearest Neighbors (k=7) Results ===')

print(f'Accuracy : {accuracy_score(y_test, y_pred_knn):.4f}')

print(f'F1 Score : {f1_score(y_test, y_pred_knn,
                             average="weighted", zero_division=0):.4f}')

print()

print('Confusion Matrix:')

print(confusion_matrix(y_test, y_pred_knn))

print()

print('Classification Report:')

print(classification_report(y_test, y_pred_knn, zero_division=0))
```

Block 15 — Confusion Matrix Heatmaps for All Three Models

```
fig, axes = plt.subplots(1, 3, figsize=(15, 4))

models_results = [

    ('Logistic Regression', y_pred_lr),

    ('Random Forest', y_pred_rf),

    ('KNN (k=7)', y_pred_knn),

]
```

```

for ax, (name, preds) in zip(axes, models_results):

    cm = confusion_matrix(y_test, preds)

    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,

                xticklabels=['Predicted 0', 'Predicted 1'],

                yticklabels=['Actual 0', 'Actual 1'])

    ax.set_title(name)

plt.suptitle('Confusion Matrices for All Three Models', fontsize=13)

plt.tight_layout()

plt.show()

```

Block 16 — Final Model Comparison Summary

```

results = {

    'Model': ['Logistic Regression', 'Random Forest', 'KNN (k=7)'],

    'Accuracy': [

        accuracy_score(y_test, y_pred_lr),

        accuracy_score(y_test, y_pred_rf),

        accuracy_score(y_test, y_pred_knn),

    ],

    'F1 Score (Weighted)': [

        f1_score(y_test, y_pred_lr, average='weighted', zero_division=0),

        f1_score(y_test, y_pred_rf, average='weighted', zero_division=0),

        f1_score(y_test, y_pred_knn, average='weighted', zero_division=0),

    ],

}

summary = pd.DataFrame(results)

summary['Accuracy'] = summary['Accuracy'].apply(lambda x: f'{x:.4f}')

summary['F1 Score (Weighted)'] = summary['F1 Score (Weighted)'].apply(lambda x: f'{x:.4f}')

print('=== Model Comparison Summary ===')

```

```
print(summary.to_string(index=False))
```

```
[37]
✓ Os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (accuracy_score, f1_score,
                             confusion_matrix, classification_report)
import warnings
warnings.filterwarnings('ignore')
```

```
[38]
✓ Os
df = pd.read_csv("/content/sample_data/Teen Mental Health Dataset.csv")
# Display settings
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
pd.set_option('display.max_colwidth', None)

print('Shape:', df.shape)
print(df.head(20).to_string())
```

Shape: (1200, 13)

	age	gender	daily_social_media_hours	platform_usage	sleep_hours	screen_time_before_sleep	academic_performance	physical_activity	social_interaction_level	stress_level
0	14	male	7.9	Instagram	7.4	2.9	3.01	1.5		low
1	19	female	1.9	TikTok	8.0	2.9	3.22	0.8		high
2	17	female	1.3	Instagram	7.6	0.5	3.92	0.0		high
3	15	male	7.4	TikTok	6.9	1.6	3.48	0.8		medium
4	15	female	4.7	Both	4.9	3.0	2.37	1.4		medium
5	19	female	7.4	Both	4.4	2.4	2.63	0.6		high

Figure 15: Full notebook running in Google Colab. all 16 cells executed